

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería en Software

**Análisis comparativo entre la Arquitectura
Orientada a Servicios (SOA) y la Arquitectura de
Microservicios:**

**Justificación de la decisión de diseño en el Sistema
de Gestión de Solicitudes de Soporte Técnico de
Internet**

Nombre: Cristhian Daniel Pacheco Cárdenas

Asignatura: Aplicaciones Distribuidas

Tipo de actividad: Trabajo autónomo individual sumativo

Período académico: Regular 2026–2027 SPA

Semestre: Séptimo Semestre

Docente: Gleiston Cicerón Guerrero Ulloa

Quevedo, Ecuador · 13 de junio de 2026

Resumen

El presente informe desarrolla un análisis comparativo entre la Arquitectura Orientada a Servicios (SOA) y la arquitectura de microservicios, con el objetivo de fundamentar la decisión de diseño adoptada en el Proyecto Fin de Curso (PFC): el Sistema de Gestión de Solicitudes de Soporte Técnico de Internet. El sistema evolucionó desde una arquitectura en tres capas esencialmente monolítica— hacia un ecosistema de cinco microservicios desacoplados, comunicados mediante REST síncrono y Apache Kafka asíncrono. Se examinan los principios fundacionales de cada estilo, sus atributos de calidad bajo el marco ISO/IEC 25010 y las condiciones concretas que determinan la aplicabilidad de uno u otro. Los hallazgos demuestran que la adopción de microservicios en lugar de SOA responde a tres restricciones verificables del proyecto: fronteras de dominio bien delimitadas por diseño guiado por el dominio (DDD), disponibilidad de infraestructura de contenedores y ausencia de sistemas heterogéneos preexistentes que justifiquen un ESB. Se concluye que los microservicios con mensajería orientada a eventos satisfacen los requisitos de escalabilidad, tolerancia a fallos y desplegabilidad independiente exigidos por el dominio de soporte técnico, mientras que SOA habría introducido complejidad sin resolver ningún problema real del sistema.

Palabras clave: arquitectura distribuida, SOA, microservicios, ESB, API Gateway, Apache Kafka, DDD, bounded context, Circuit Breaker, Saga, atributos de calidad, soporte técnico.

Índice

1. Introducción	3
1.1. Objetivo general	3
1.2. Objetivos específicos	3
1.3. Alcance	4
2. Marco teórico	4
2.1. Arquitectura de software de un sistema distribuido	4
2.2. El monolito en capas como punto de partida	4
2.3. Arquitectura Orientada a Servicios	5
2.4. Arquitectura de microservicios	5
2.5. La Arquitectura Orientada a Eventos como patrón complementario . .	6
3. Análisis comparativo: SOA vs. microservicios	6
3.1. Criterios de comparación	6
3.2. Unidad de despliegue	6
3.3. Escalabilidad	7
3.4. Gestión de datos	7
3.5. Modelo de comunicación	7
3.6. Tolerancia a fallos	8
3.7. Complejidad operativa	8
3.8. Síntesis comparativa	8
3.9. Análisis crítico	9
4. Aplicación al Proyecto Fin de Curso	9
4.1. Descripción del sistema	9
4.2. Evolución arquitectónica: del monolito a los microservicios	10
4.3. Identificación de bounded contexts mediante DDD	10
4.4. Catálogo de microservicios adoptados	11
4.5. Patrones de microservicios aplicados	11
4.6. Por qué no SOA: justificación técnica	12
4.7. Limitaciones reconocidas y medidas de mitigación	12
5. Conclusiones	13

1. Introducción

El diseño de sistemas distribuidos plantea una de las decisiones más determinantes en ingeniería de software: elegir el estilo arquitectónico sobre el que se construirá el sistema. Esa decisión no es fácilmente reversible una vez que el código existe y el equipo se ha organizado alrededor de un modelo de trabajo. Por eso, tomarla con criterios técnicos verificables y no con base en tendencias de mercado es una responsabilidad central del arquitecto de software.

El Sistema de Gestión de Solicitudes de Soporte Técnico de Internet denominado en adelante *el sistema* o PFC nació como una aplicación web monolítica en tres capas. Esta arquitectura inicial tiene ventajas reconocibles: es simple de razonar, fácil de depurar y suficiente para un equipo pequeño en fase de exploración del dominio. Sin embargo, a medida que el sistema incorpora un componente de inteligencia artificial para la clasificación automática de incidencias, notificaciones multicanal y reportes analíticos, la arquitectura monolítica se convierte en una limitación estructural [1].

La pregunta que motiva este informe no es si migrar a una arquitectura más distribuida esa decisión ya está tomada en el PFC sino cuál de los dos estilos candidatos responde mejor a las necesidades concretas del sistema: la Arquitectura Orientada a Servicios (SOA) o la arquitectura de microservicios. Ambas proponen distribuir la lógica en unidades de negocio autónomas; ambas buscan bajo acoplamiento y alta cohesión. Pero difieren profundamente en cómo logran esos objetivos y qué precio operativo cobran por ellos [2].

SOA integra servicios a través de un Bus de Servicios Empresariales (ESB) que centraliza el enrutamiento, la transformación de mensajes y la orquestación de procesos. Los microservicios, en cambio, distribuyen esa responsabilidad: cada servicio decide cómo comunicarse con los demás, sin intermediario central, usando protocolos ligeros y mensajería asíncrona [3]. La diferencia parece sutil en el papel, pero sus implicaciones operativas son radicales en la práctica.

1.1. Objetivo general

Comparar de manera crítica los estilos arquitectónicos SOA y microservicios para fundamentar la decisión de diseño adoptada en el Sistema de Gestión de Solicitudes de Soporte Técnico de Internet.

1.2. Objetivos específicos

1. Caracterizar los principios fundacionales de SOA y microservicios a partir de literatura académica verificada.
2. Contrastar ambas arquitecturas en criterios de escalabilidad, despliegue, gestión de datos, comunicación y tolerancia a fallos.

3. Proyectar los hallazgos del análisis sobre el contexto real del PFC, identificando cuál estilo resuelve mejor sus restricciones verificables.
4. Reconocer las limitaciones de la arquitectura adoptada y proponer medidas de mitigación concretas.

1.3. Alcance

El análisis cubre los estilos SOA y microservicios en su dimensión de diseño lógico. No evalúa infraestructura de red, hardware ni proveedores de nube específicos. La proyección al PFC toma como referencia la Entrega 2 del sistema, que establece el diseño de arquitectura de microservicios con cinco servicios independientes, comunicados mediante REST y Apache Kafka.

2. Marco teórico

2.1. Arquitectura de software de un sistema distribuido

La arquitectura de software de un sistema distribuido es el conjunto de **decisiones estructurales** que determinan qué componentes existen, qué responsabilidad tiene cada uno y cómo se comunican, con independencia del hardware donde se ejecuten. Esta concepción distingue dos vistas: la *vista física* nodos, redes y protocolos de transporte y la *vista lógica* componentes, módulos, servicios, contratos e interfaces.

El presente análisis trabaja sobre la vista lógica porque es la que determina las propiedades de mantenibilidad, escalabilidad y despleabilidad. Como señalan Shadija, Rezai y Hill [2], la arquitectura lógica de un sistema distribuido condiciona directamente sus atributos de calidad bajo el marco ISO/IEC 25010: escalabilidad, disponibilidad, tolerancia a fallos y despleabilidad.

2.2. El monolito en capas como punto de partida

Antes de comparar SOA y microservicios, es necesario situar el punto de origen del sistema: una arquitectura en tres capas compuesta por una capa de presentación la interfaz web, una capa de lógica de negocio validaciones, flujos de asignación y gestión de roles y una capa de datos el repositorio relacional con toda la información del sistema. Las tres capas se despliegan como una sola unidad ejecutable sobre una base de datos compartida.

Las ventajas de este punto de partida son reales: simplicidad inicial, transacciones locales con consistencia inmediata, depuración de extremo a extremo sencilla y latencia mínima entre capas porque las llamadas ocurren en memoria [1]. Sin embargo, a medida que el sistema crece incorporando inteligencia artificial, reportes analíticos y notificaciones multicanal el monolito acumula tres problemas estructurales: el escalado es en bloque, cualquier cambio requiere rediseñar toda la aplicación y la base de datos compartida impide la evolución independiente de los módulos.

2.3. Arquitectura Orientada a Servicios

SOA es un estilo en el que las capacidades de negocio se exponen como **servicios reutilizables** con contratos formales típicamente WSDL y mensajes SOAP integrados a través de un Bus de Servicios Empresariales (ESB). El ESB actúa como intermediario central que enruta, transforma y orquesta mensajes entre los distintos sistemas de la organización.

Sus principios fundamentales, documentados por Shadija et al. [2], incluyen: acoplamiento débil entre servicios, contrato de servicio estandarizado, abstracción de la implementación, reutilización y autonomía relativa. En la práctica, SOA fue adoptado masivamente en entornos empresariales durante la primera década del siglo XXI para resolver el problema de integración entre sistemas heterogéneos ERPs, CRMs y aplicaciones legadas sin necesidad de reescribirlos.

El riesgo histórico de SOA reside precisamente en el ESB: al concentrar lógica de enrutamiento, transformación de mensajes y orquestación de procesos, el bus se convierte en un componente de alta complejidad interna. Cuando el ESB falla o se satura, la comunicación de toda la organización se interrumpe de forma simultánea. Liu et al. [3] identifican este punto único de fallo como la razón principal por la que SOA fue desplazado por los microservicios en contextos de alta disponibilidad.

2.4. Arquitectura de microservicios

Los microservicios son, según Shadija et al. [2], una forma particular de SOA aplicada al nivel de la aplicación en lugar del nivel empresarial. La diferencia fundamental no está en los principios ambos buscan servicios con contratos y bajo acoplamiento sino en la granularidad, el mecanismo de integración y la cultura de equipo requerida.

Liu et al. [3] definen la arquitectura de microservicios como un paradigma que descompone una aplicación en **unidades funcionales pequeñas e independientes**, cada una responsable de una capacidad de negocio concreta, ejecutándose en su propio proceso y comunicándose mediante mecanismos ligeros. Sus propiedades clave son: modelado alrededor de una capacidad de negocio no de una capa técnica, despliegue independiente, datos descentralizados con cada servicio como propietario exclusivo de su base de datos, y tolerancia a fallos mediante aislamiento de procesos.

Blinowski et al. [1] advierten que los microservicios eliminan el ESB central distribuyendo la inteligencia de comunicación en los propios servicios, lo que reduce el acoplamiento estructural al precio de incrementar la complejidad operativa: los equipos deben gestionar observabilidad distribuida, descubrimiento de servicios, resiliencia de red y consistencia eventual. Esta advertencia es directamente relevante para el PFC: adoptar microservicios sin infraestructura de contenedores equivale a crear un *monolito distribuido* lo peor de ambos mundos.

2.5. La Arquitectura Orientada a Eventos como patrón complementario

La Arquitectura Orientada a Eventos (EDA) actúa como complemento natural de los microservicios. El productor publica un evento en un intermediario de mensajes (*broker*) como Apache Kafka y continúa trabajando sin esperar respuesta. Los consumidores se suscriben y reaccionan al evento a su propio ritmo, produciendo un desacoplamiento temporal entre servicios.

Rahmatulloh et al. [4] demuestran empíricamente que integrar EDA dentro de una arquitectura de microservicios mejora el tiempo de respuesta en un 19,18 % y reduce la tasa de error en un 34,40 % frente a la comunicación HTTP síncrona, si bien incrementa el uso de CPU en un 8,52 %. Este hallazgo es directamente aplicable al PFC, donde Kafka conecta de forma asíncrona el servicio de tickets con el de inteligencia artificial y el de notificaciones, permitiendo que el cliente reciba el identificador del ticket de forma inmediata sin esperar a que la IA complete la clasificación.

El estudio de caso de Laigner et al. [5] migración real de un sistema monolítico de Big Data a microservicios con EDA confirma que este patrón facilita el mantenimiento y el aislamiento de fallos en sistemas con múltiples actores y flujos de trabajo complejos, que es exactamente el escenario del PFC.

3. Análisis comparativo: SOA vs. microservicios

3.1. Criterios de comparación

El análisis se estructura en torno a seis dimensiones técnicas extraídas de la literatura revisada y alineadas con los atributos de calidad ISO/IEC 25010: unidad de despliegue, escalabilidad, gestión de datos, modelo de comunicación, tolerancia a fallos y complejidad operativa.

3.2. Unidad de despliegue

En SOA, la unidad de despliegue es el servicio empresarial, un componente de tamaño mediano o grande que depende del ESB para ser invocado. Actualizar un servicio implica coordinar su nueva versión con el bus y verificar compatibilidad con todos sus consumidores registrados, lo que frena la velocidad de entrega.

En microservicios, cada unidad de despliegue es un proceso independiente. Esto permite publicar cambios en el servicio de notificaciones sin detener ni modificar el servicio de tickets ni el de autenticación. Laigner et al. [5] confirman esta ventaja en un estudio de caso real: la migración a microservicios con EDA facilitó el mantenimiento y el aislamiento de fallos de manera notable, aunque la diversidad tecnológica incorporada fue identificada como una desventaja operativa concreta.

3.3. Escalabilidad

SOA permite escalar servicios individuales, pero el ESB actúa como limitante: si el bus se satura, la capacidad de los servicios independientes queda subutilizada. Los microservicios permiten lo que Blinowski et al. [1] denominan *escalado fino*: solo el servicio saturado se replica, sin necesidad de escalar el sistema completo.

No obstante, los mismos autores formulan una advertencia crítica: para sistemas que no manejan miles de usuarios concurrentes, los beneficios del escalado en microservicios no han sido investigados con rigor suficiente y la evidencia disponible es inconsistente [1]. Adoptar microservicios en un sistema de tráfico moderado puede resultar en mayor complejidad operativa sin ningún beneficio real de rendimiento, advertencia que el equipo del PFC debe tener presente durante la fase de implementación.

3.4. Gestión de datos

Esta dimensión representa la diferencia más profunda entre ambos estilos. En SOA, la práctica habitual consiste en compartir una base de datos centralizada entre múltiples servicios, lo que simplifica las consultas cruzadas mediante sentencias JOIN pero genera acoplamiento de datos que impide la evolución independiente.

En microservicios, el patrón *Database per Service* es un principio fundacional: cada servicio es propietario exclusivo de sus datos y ningún otro puede consultarlos directamente. Los JOIN entre servicios deben reemplazarse por composición de llamadas REST o eventos. Para gestionar transacciones distribuidas se emplean los patrones CQRS Segregación de Responsabilidad entre Comandos y Consultas y Saga secuencia de transacciones locales con compensaciones ante fallo.

3.5. Modelo de comunicación

SOA utiliza predominantemente protocolos pesados: SOAP sobre HTTP con mensajes XML y contratos WSDL. La transformación y el enrutamiento de mensajes ocurren dentro del ESB, lo que introduce latencia adicional y rigidez contractual.

Los microservicios favorecen protocolos ligeros: REST sobre HTTP para APIs públicas, gRPC para comunicación interna de alta frecuencia, y mensajería asíncrona para eventos y notificaciones. Liu et al. [3] consolidan la regla práctica de la industria: REST hacia afuera, gRPC o mensajería hacia adentro, y eventos para todo aquello que pueda tolerar latencia.

3.6. Tolerancia a fallos

En SOA, el ESB constituye un punto único de fallo de alto impacto: si el bus cae, todos los servicios pierden su canal de comunicación simultáneamente. En microservicios, la tolerancia a fallos se construye mediante patrones específicos: *circuit breaker* cortacircuito que deja de enviar peticiones a un servicio degradado, reintentos con retroceso exponencial y tiempos de espera explícitos. El fallo de un servicio se manifiesta como degradación controlada del sistema en lugar de un colapso general.

3.7. Complejidad operativa

SOA presenta complejidad centralizada en el ESB: configuración de rutas, políticas de transformación y gobierno de servicios. Los microservicios presentan complejidad alta y distribuida: contenedores, observabilidad, descubrimiento de servicios y CI/CD. Liu et al. [3] identifican la depuración entre servicios, la latencia acumulada y la consistencia de datos distribuidos como los desafíos más críticos en producción.

3.8. Síntesis comparativa

La Tabla 1 consolida los resultados del análisis.

Tabla 1: Comparativa técnica entre SOA y microservicios

Criterio	SOA	Microservicios
Unidad de despliegue	Servicio grande coordinado con el ESB	Proceso pequeño e independiente
Escalabilidad	Por servicio, limitada por el ESB	Granular: solo el servicio saturado
Gestión de datos	Base de datos frecuentemente compartida	Una base por servicio (<i>Database per Service</i>)
Comunicación	SOAP/XML enrutado por el ESB	REST, gRPC, mensajería asíncrona
Tolerancia a fallos	ESB como punto único de fallo	Aislamiento por servicio (<i>circuit breaker</i>)
Complejidad operativa	Alta, centralizada en el ESB	Alta, distribuida: contenedores, CI/CD, observabilidad
Contexto adecuado	Integración de sistemas heterogéneos empresariales preexistentes	Equipos autónomos, dominio maduro, despliegue frecuente

3.9. Análisis crítico

La Tabla 1 revela que SOA y microservicios son respuestas a problemas distintos en escalas distintas. SOA resuelve la *interoperabilidad empresarial*: cómo hacer que sistemas heterogéneos escritos décadas antes intercambien información sin reescribirlos. Los microservicios resuelven la *velocidad de entrega y escalabilidad autónoma*: cómo permitir que equipos independientes desplieguen y escalen sus componentes sin coordinación central.

Un error conceptual frecuente es evaluar los microservicios como una versión mejorada de SOA. Shadija et al. [2] aclaran que los microservicios son SOA aplicado al nivel de la aplicación. La diferencia no está en los principios sino en la granularidad, el mecanismo de integración y la cultura de equipo: SOA centraliza la inteligencia en el bus, los microservicios la distribuyen en los propios servicios.

La limitación más crítica de los microservicios, señalada con rigor por Blinowski et al. [1], es que para sistemas de escala moderada la complejidad operativa puede superar los beneficios. Un equipo que implementa microservicios sin contenedores ni observabilidad no obtiene despliegue independiente: obtiene un sistema distribuido frágil y costoso de mantener. Este riesgo es concreto en proyectos académicos de equipos de cuatro personas y debe gestionarse con disciplina.

SOA, por su parte, tiene una limitación estructural igualmente seria: el ESB concentra inteligencia en una capa que no pertenece a ningún dominio de negocio, y cuando falla, toda la organización lo siente al mismo tiempo. Esa fragilidad fue, históricamente, la razón que motivó el surgimiento de los microservicios.

4. Aplicación al Proyecto Fin de Curso

4.1. Descripción del sistema

El Sistema de Gestión de Solicitudes de Soporte Técnico de Internet es una plataforma web orientada a formalizar el proceso de atención de incidencias relacionadas con la conectividad dentro de entornos organizacionales. El sistema opera mediante un esquema de acceso restringido por roles: el *cliente* registra solicitudes y realiza seguimiento; el *técnico* atiende los casos asignados; el *administrador* supervisa el sistema y asigna técnicos; el *superusuario* gestiona cuentas y perfiles.

El dominio central es el ciclo de vida completo de un ticket: desde que el cliente reporta una incidencia pérdida de conexión, degradación de velocidad, fallo de hardware de red hasta la resolución confirmada y el cierre formal, pasando por la asignación a técnicos, el registro de trámites, las notificaciones de estado y el análisis inteligente de patrones de falla.

El proyecto es desarrollado por un equipo de cuatro personas con roles definidos: arquitecto responsable del diseño distribuido, líder de desarrollo responsable de la implementación de los servicios críticos, responsable de calidad a cargo de las pruebas de integración y responsable de documentación.

4.2. Evolución arquitectónica: del monolito a los microservicios

La versión inicial del sistema se construyó sobre una arquitectura en tres capas: capa de presentación, capa de lógica de negocio y capa de datos. Esta arquitectura, apropiada para la fase de exploración del dominio, presentó limitaciones concretas a medida que el sistema incorporó capacidades adicionales:

- El componente de inteligencia artificial para clasificación automática de tickets requiere un proceso independiente con dependencias Python, incompatibles con el entorno Java del sistema principal.
- El servicio de notificaciones multicanal debe poder desplegarse y escalarse de forma independiente, ya que su latencia no debe bloquear la creación de solicitudes por parte del cliente.
- El módulo de reportes analíticos necesita un modelo de datos desnormalizado para consultas complejas, incompatible con la base de datos operacional compartida.

Estas tres necesidades concretas no una tendencia tecnológica justificaron la migración a microservicios en la Entrega 2 del PFC.

4.3. Identificación de bounded contexts mediante DDD

La descomposición en microservicios siguió los principios del Diseño Guiado por el Dominio (DDD). Se identificaron cinco *bounded contexts* diferenciados por su responsabilidad:

- **BC-1: Autenticación y Autorización.** Gestiona la identidad de todos los actores, emite y valida tokens JWT y controla permisos basados en roles (RBAC). Actúa como proveedor de identidad para todos los demás.
- **BC-2: Gestión de Tickets.** Centraliza el ciclo de vida completo de una solicitud de soporte. Es el núcleo del dominio (*core domain*).
- **BC-3: Notificaciones.** Consume eventos del bus de mensajes y distribuye notificaciones según el canal configurado.
- **BC-4: Inteligencia Artificial.** Clasifica automáticamente las incidencias, estima prioridad y sugiere técnicos basándose en historial.
- **BC-5: Reportes y Análisis.** Construye y expone métricas operativas: tiempo promedio de resolución, cumplimiento de SLA y volumen de incidencias.

4.4. Catálogo de microservicios adoptados

La Tabla 2 describe el catálogo completo de microservicios del sistema resultante de la migración.

Tabla 2: Microservicios del Sistema de Gestión de Soporte Técnico

Servicio	Puerto	Base de datos	Responsabilidad principal
auth-service	8001	PostgreSQL	Registro, login, JWT y RBAC
ticket-service	8002	PostgreSQL	CRUD de tickets, estados, asignaciones y SLA
notification-service	8003	MongoDB	Consumo de eventos Kafka, despacho multicanal
ai-service	8004	MongoDB	Clasificación NLP, priorización y sugerencias
report-service	8005	PostgreSQL	Modelo CQRS de lectura, métricas y exportación

El API Gateway (Spring Cloud Gateway, puerto 8080) actúa como punto único de entrada para todos los clientes externos: enruta peticiones, verifica tokens JWT delegando al `auth-service`, aplica límites de tasa y centraliza el registro de accesos. Apache Kafka opera como bus de mensajes asíncrono para los eventos de dominio entre servicios.

4.5. Patrones de microservicios aplicados

El sistema implementa tres patrones que responden a desafíos concretos del dominio:

CQRS: El `ticket-service` recibe un alto volumen de escrituras pero el `report-service` necesita consultas complejas con agregaciones que penalizarían la base de datos operacional. La solución separa el modelo de escritura del modelo de lectura: el `ticket-service` actualiza su propia base de datos; el `report-service` mantiene una base de datos desnormalizada que se actualiza escuchando los eventos Kafka del `ticket-service`.

Saga por coreografía. La creación de un ticket involucra múltiples servicios en cadena. El `ticket-service` crea el registro en estado `PENDIENTE_CLASIFICACION` y publica `ticket.created`; el `ai-service` clasifica la incidencia y publica `ticket.classified`; el `ticket-service` actualiza categoría y prioridad y publica `ticket.ready`; el `notification-service` envía la confirmación al cliente. Si la clasificación falla, el `ai-service` publica un evento de compensación y el ticket queda en estado de error, sin afectar a los demás servicios.

Este flujo implementa exactamente el desacoplamiento temporal que Rahmatulloh et al. [4] demuestran que mejora el rendimiento y reduce la tasa de errores.

Circuit Breaker: Si el `ticket-service` se degrada, el API Gateway acumularía peticiones en espera hasta agotar su pool de hilos, dejando inoperables incluso las rutas de autenticación. El Circuit Breaker implementado con Resilience4j abre el circuito al superar el 50 % de tasa de fallo en las últimas diez peticiones y devuelve inmediatamente una respuesta controlada sin propagar el timeout al cliente mientras los demás microservicios continúan operando.

4.6. Por qué no SOA: justificación técnica

El rechazo de SOA como alternativa se fundamenta en tres razones técnicas verificables derivadas del análisis de la Sección 3:

Primera: el sistema se construye desde cero con tecnología seleccionada por el equipo. No existe ningún sistema legado heterogéneo que requiera un ESB para integrarse. Introducir un ESB en este contexto equivale a crear artificialmente el problema que SOA fue diseñado para resolver.

Segunda: un ESB centralizado contradice el requisito de disponibilidad del sistema de soporte. Si el bus falla, la comunicación de todos los servicios se interrumpe simultáneamente. La arquitectura de microservicios con Kafka y Circuit Breaker garantiza que el fallo de un servicio incluso el de inteligencia artificial se degrada de forma controlada sin colapsar el sistema completo.

Tercera: los cinco bounded contexts identificados mediante DDD tienen fronteras suficientemente claras como para justificar servicios autónomos. No existe coordinación constante entre ellos que requiera un orquestador central.

4.7. Limitaciones reconocidas y medidas de mitigación

El equipo reconoce tres limitaciones técnicas concretas:

1. **Consistencia eventual.** Las operaciones que involucran más de un servicio no pueden garantizar consistencia inmediata. El patrón Saga mitiga este problema, pero introduce lógica de compensación que debe diseñarse y probarse con cuidado. Laigner et al. [5] confirman que este costo fue uno de los más significativos en su migración real. El equipo debe documentar los estados transitorios aceptables y los tiempos máximos de propagación de cada evento.

2. **Complejidad de depuración distribuida.** Los errores que atraviesan múltiples servicios son más difíciles de rastrear que en un monolito. Como mitigación, el equipo debe incorporar identificadores de correlación en los encabezados HTTP y en los metadatos de cada evento Kafka, siguiendo la recomendación de Liu et al. [3], de modo que los logs de los cinco servicios puedan correlacionarse durante la depuración.
3. **Carga operativa por diversidad tecnológica.** El sistema integra Spring Boot (Java), Node.js y FastAPI (Python), con bases de datos PostgreSQL y MongoDB. Esta heterogeneidad es una ventaja arquitectónica cada servicio usa la tecnología más adecuada pero una carga real para un equipo de cuatro personas. Laigner et al. [5] documentaron exactamente este trade-off en su caso de estudio.

5. Conclusiones

El análisis comparativo entre SOA y microservicios confirma que ninguno de los dos estilos es universalmente superior: son respuestas a problemas distintos en contextos distintos. SOA resuelve la interoperabilidad entre sistemas heterogéneos preexistentes mediante contratos formales y un bus de orquestación; los microservicios resuelven la velocidad de entrega y la escalabilidad granular mediante servicios autónomos y comunicación ligera.

Para el Sistema de Gestión de Solicitudes de Soporte Técnico de Internet, la elección de microservicios es la correcta por tres razones verificables: el dominio tiene cinco fronteras de negocio limpias identificadas mediante DDD, la infraestructura de contenedores está disponible con Docker y Docker Compose, y no existe ningún sistema legado que requiera un ESB para integrarse. SOA habría introducido complejidad de configuración sin resolver ningún problema real del PFC.

Sin embargo, el equipo debe asumir conscientemente los costos de la decisión: consistencia eventual en operaciones distribuidas, complejidad de depuración entre servicios y carga operativa de mantener múltiples pilas tecnológicas. Estos costos no son argumentos en contra de la arquitectura adoptada; son restricciones de diseño que deben gestionarse desde las primeras iteraciones con patrones específicos Saga, identificadores de correlación, builds multi-stage y no descubrir en producción.

La enseñanza más importante del análisis, respaldada por Blinowski et al. [1], es que los beneficios de los microservicios no son automáticos. Solo se materializan cuando el equipo tiene la disciplina operativa necesaria y el dominio tiene las fronteras claras que permiten que la autonomía de los servicios sea real. En el PFC, ambas condiciones están presentes. En sistemas donde no lo están, el monolito modular sigue siendo la decisión más honesta.

Repositorio en GitHub

El código en LaTeX de este documento se encuentra disponible en:

<https://github.com/CristhianP03/guia01-socketsv01/blob/main/Informe%20Acad%C3%A9mico%20Individual%20%E2%80%94%20An%C3%A1lisis%20Comparativo%20de%20Arquitectura%20Distribuidas/C%C3%B3digo%20LaTeX.tex>

Rol de la Inteligencia Artificial

Durante la elaboración de este informe se utilizó una herramienta de inteligencia artificial como apoyo en la redacción. Su participación se limitó a correcciones menores de texto y estructura, como mejorar la redacción de algunos párrafos, reorganizar el orden de ciertas explicaciones y unificar el estilo del documento.

Referencias bibliográficas

- [1] G. Blinowski, A. Ojdowska y A. Przybyłek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803.
- [2] D. Shadija, M. Rezai y R. Hill, “Towards an Understanding of Microservices,” en *Proc. 2017 23rd Int. Conf. Automation and Computing (ICAC)*, Huddersfield, United Kingdom: IEEE, 2017, págs. 308-313. DOI: 10.23919/ICAC.2017.8082018.
- [3] G. Liu, B. Huang, Z. Liang, M. Qin, H. Zhou y Z. Li, “Microservices: Architecture, Container, and Challenges,” en *Proc. 2020 IEEE 20th Int. Conf. Software Quality, Reliability and Security Companion (QRS-C)*, Macau, China: IEEE, 2020, págs. 629-635. DOI: 10.1109/QRS-C51114.2020.00107.
- [4] A. Rahmatulloh, F. Nugraha, R. Gunawan e I. Darmawan, “Event-Driven Architecture to Improve Performance and Scalability in Microservices-Based Systems,” en *Proc. 2022 Int. Conf. Advancement in Data Science, E-Learning and Information Systems (ICADEIS)*, Bandung, Indonesia: IEEE, 2022, págs. 1-6. DOI: 10.1109/ICADEIS56544.2022.10037390.
- [5] R. Laigner et al., “From a Monolithic Big Data System to a Microservices Event-Driven Architecture,” en *Proc. 2020 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, Portořož, Slovenia: IEEE, 2020, págs. 213-220. DOI: 10.1109/SEAA51224.2020.00045.